

OJ 实验报告

史锐瑞 2025010531

FastAPI（后端）+Streamlit（前端）+JSON 文件持久化

一、系统功能与设计

1.1 系统架构

系统采用前后端分离架构，通过 RESTAPI 通信：

后端 OJ.py：FastAPI 编写的异步 Web 服务，负责业务逻辑、权限控制、代码评测，监听 127.0.0.1:8000。

前端 app.py：Streamlit 编写的图形界面，只通过 HTTP 调用后端 API，不直接读写任何数据文件。

数据层：全部用 JSON 文件本地持久化，无外部数据库，结构清晰、便于评审。

1.2 主要功能

系统按官方文档的 Step1 – 6+Advance 逐步实现：

阶段	功能	说明
Step1	题目管理	题目的新增、查看、编辑、删除，字段完整校验
Step2	评测	提交代码，编译+运行+逐测试点比对判定
Step3	列表/重测	提交记录分页筛选，管理员重新评测
Step4	用户管理	注册、登录、改密、改名、角色 (user/admin/banned)
Step5	日志权限	评测日志可见性控制+访问审计
Step6	前端	Streamlit 图形界面，覆盖所有接口
Advance	AI 智能命题	调用大模型自动生成题目，可一键入库

1.3 技术选型

类别	选择	理由
Web 框架	FastAPI	原生 asyncdef 支持异步评测，自动 OpenAPI 文档
前端	Streamlit	纯 Python 快速构建交互界面
密码存储	bcrypt	bcrypt 加密，登录时校验
AI 调用	httpx	支持流式 aiter_lines()，异步非阻塞
评测执行	subprocess	真正编译运行用户代码，非模拟
存储	JSON 文件	免去数据库依赖，便于评审与迁移

1.4 模块划分

后端核心模块 (OJ.py)：

1. 统一响应层——所有接口返回 {code,msg,data}，HTTP 状态码与 code 一致：

```
def ok(data: Any = None, msg: str = "success") -> JSONResponse:
    return JSONResponse(status_code=200,
content={"code": 200, "msg": msg, "data": data})

def fail(code: int, msg: str) -> JSONResponse:
    return JSONResponse(status_code=code,
content={"code": code, "msg": msg, "data": None})
```

2. 题目校验层——normalize_problem()校验必填/可选字段、补全默认值，并用正则限制 id 防止文件名注入

3. 鉴权层——_require_user()/_require_admin()两个中间函数统一做登录与权限判断，banned 用户直接拒绝：

```
def _require_admin(request):
    user, err = _require_user(request)
```

```

if err is not None:
    return None, err

if user.get("role") != "admin":
    return None, fail(403, "权限不足")

return user, None

```

4. 评测引擎——`_judge_submission()`后台任务+`_run_process()`子进程封装

5. AI 命题模块——流式调用、JSON 提取、失败自动修正、中断与计价（详见第二章）。

前端模块（`app.py`）：

统一 `api()` 封装，集中处理 Cookie 同步、后端不可达提示、会话过期自动登出：

```

def api(method, path, json=None, params=None):
    resp = requests.request(method, BASE_URL + path,
json=json, params=params,
                                cookies=_cookies(),
timeout=15)
    ...
    token = resp.cookies.get(SESSION_COOKIE)
    if token:
        st.session_state["session_token"] = token

```

五大页面：用户中心、题目管理、评测与提交、语言管理、AI 智能命题，通过侧边栏 radio 导航。

二、关键实现与难点

2.1 评测执行引擎

评测的核心是真实编译运行用户代码，再逐测试点比对输出。整个评测在 `asyncio.create_task` 派生的后台任务中执行，避免阻塞事件循环；阻塞的 `subprocess.run` 用 `asyncio.to_thread` 丢进线程池：

```
def _target():
    proc = subprocess.run(cmd, cwd=str(cwd),
input=stdin_data.encode(),
                                stdout=subprocess.PIPE,
stderr=subprocess.PIPE, timeout=timeout)
    return (proc.returncode,
proc.stdout.decode(...), proc.stderr.decode(...),
False, _peak_memory_mb())
return await asyncio.to_thread(_target)
```

编译阶段：编译型语言先执行 `compile_cmd`，编译失败直接记 `compile_info` 得 0 分；运行阶段逐测试点判定：

```
for idx, tc in enumerate(testcases, start=1):
    rc, out, err, timed, mem = await
_run_process(run_cmd, workdir, tc.get("input",
""), ...)
    if timed:
        result = "TLE"
```

```

elif rc != 0:
    result = "MLE" if _is_mle(rc, err) else "RE"
else:
    result = "AC" if _normalize_output(out) ==
_normalize_output(tc.get("output", "")) else "WA"
    details.append({"id": idx, "result": result,
"time": ..., "memory": mem})
    if result == "AC":
        score += 10

```

2.2 输出比对（难点：忽略末尾空白）

用户程序输出末尾多一个换行、行末多空格是常见现象，直接字符串比较会误判 WA。解法是先规范化再比较：

```

def _normalize_output(s: str) -> list[str]:
    lines = s.replace("\r\n", "\n").split("\n")
    lines = [line.rstrip() for line in lines] # 去掉每行行末空格

    while lines and lines[-1] == "":
        lines.pop() # 去掉末尾多余空行

    return lines

```

2.3 命令模板与跨平台路径（难点）

语言配置用命令模板（如 `g++{src}-o{exe}`），运行时把 `{src}/{exe}` 替换成临

时工作目录下的实际路径。Windows 路径含反斜杠会破坏 shell 解析，故统一替换为正斜杠并用 shlex.split 安全拆分：

```
def _build_command(template, src, exe):  
    def _p(p): return str(p).replace("\\", "/")  
    return shlex.split(template.replace("{src}",  
_p(src)).replace("{exe}", _p(exe)))
```

2.4 内存与时间限制（难点+已知局限）

时间限制靠 subprocess.run(timeout=...)实现；内存限制在 POSIX 下用

RLIMIT_AS 限制地址空间、用 getrusage 统计峰值：

```
def _preexec():  
    resource.setrlimit(resource.RLIMIT_AS, (limit,  
limit))
```

难点与局限：RLIMIT_AS 只在 Linux/macOS 生效，Windows 上内存限制不真正执行（代码已做平台判断，功能不报错，但内存约束失效）。这是本项目在 Windows 环境下评测的一个已知短板。

2.5 权限与审计（Step5）

评测日志的可见性遵循三级规则：管理员、提交本人、以及题目开启了

public_cases 的用户；每次日志访问都会写入审计记录，供管理员追溯：

```
allowed = (user.get("role") == "admin"  
           or sub.get("user_id") ==  
user.get("user_id")  
           or is_log_public(problem_id))
```

```
_append_access_audit(user["user_id"], problem_id,  
"200" if allowed else "403")
```

2.6AI 智能命题 (Advance, 重点难点)

调用 OpenAI 兼容接口流式生成题目，难点集中在四点：

①容错解析 JSON——模型输出常带 json 代码块或前后废话，用「栈式括号配平」提取首个完整 JSON：

```
def _extract_json(text):  
    fence = re.search(r"```(?:json)?\s*(.*?)```",  
text, re.S)  
    if fence: text = fence.group(1).strip()  
    start = text.find("{")  
    depth = 0  
    for i in range(start, len(text)):  
        if text[i] == "{": depth += 1  
        elif text[i] == "}":  
            depth -= 1  
            if depth == 0:  
                return json.loads(text[start:i + 1])
```

②失败自动修正——首次解析失败时，把错误信息回传给模型再请求一次，让模型自己修正输出格式。

③流式中断——用 `cancel_requested` 标志，在流式循环里每收到一个 chunk 就检查，命中即停止请求并落盘 `cancelled` 状态。

④密钥不泄露——api_key 只存本地，任何查询响应只返回布尔标记

api_key_configured:

```
def _safe_model_config():  
    return { ..., "api_key_configured":  
bool(_model_config.get("api_key")) }
```

2.7 难点与解决方案汇总

难点	解决方案
后台评测任务被取消 (TestClient 会取消 create_task)	自测时用真实 uvicorn 服务验证后台任务
输出末尾空白导致误判	_normalize_output 规范化后比对
Windows 路径反斜杠破坏命令	统一正斜杠+shlex.split
内存限制跨平台	POSIX 用 RLIMIT_AS, Windows 降级为不限制并注释说明
模型输出不是合法 JSON	栈式配平提取+失败自动修正一次
APIkey 泄露风险	响应只返回布尔标记
提交洪泛	内存队列实现「1 分钟最多 3 次」频率限制

三、成果展示

3.1 系统效果

题库内置 21 道题目 ([problems/](#))，覆盖入门输入输出、数学、字符串、数组、二分、动态规划等知识点。

前端五大页面全部可用，管理员与非管理员权限区分正确：普通用户只能看题、做题，题目增/删/改仅管理员。

3.2 边界测试结果

针对评测判定各分支做了边界覆盖：

测试项	输入/场景	预期	结果
常规 AC	sum4 组测试点 (含-55、大数 10000000001000000000)	全 AC	✓
大数范围	10000000001000000000→2000000000 (超出 32 位)	正确	✓
输出多余空白	输出末尾多换行/行末空格	不误判 WA	✓
编译错误	C++语法错误提交	compile_info.result=error, 0 分	✓
超时 TLE	死循环代码	判 TLE	✓
运行错误 RE	除零/段错误	判 RE	✓
权限	普通用户删除/编辑题目	403 拒绝	✓
频率限制	1 分钟内第 4 次提交	429	✓
AI 命题	非法模型名 ds	返回 400, 提示可用模型名	✓

四、AI 使用说明

4.1 工具链

ClaudeCode (VibeCoding 模式) 作为主力开发工具, 负责代码生成、接口联调与问题修复。

后端接口对齐官方 API 文档 (<https://dbg-course.github.io/python-docs/oj/>)。

辅助工具: curl 调试接口、uvicorn 起服务自测、requests 轮询验证后台任务。

4.2 workflow

1. 需求拆解: 按官方文档把 Step1 - 6+Advance 拆成独立模块与接口清单。
2. AI 生成骨架: 让 AI 按接口清单生成后端路由、前端页面框架。
3. 人工验收: 逐条用真实服务跑接口, 核对返回结构、状态码、错误顺序是否符合 api.md。
4. 难点攻坚: 评测判定、内存限制、AI 流式生成等关键逻辑, 由 AI 提出

方案、人工验证后落地，问题（如后台任务被 TestClient 取消）通过排查定位并修复。

4.3 VibeCoding 代码比例

约 70% - 80% 的代码由 AI 生成或辅助完成（含后端接口、前端页面、校验逻辑），其余约 20% - 30% 为人工编写与调优——主要集中在评测引擎的边界处理、权限规则的梳理确认，以及对 AI 生成代码的逐条验证和错误修正。

五、总结与建议

5.1 收获

完整走通了一个前后端分离系统的设计与实现，理解了异步 Web 框架（FastAPI）与后台任务（`asyncio.create_task`）的协作方式。深入实践了代码评测的核心：编译、运行、输出比对、资源限制，以及跨平台差异。学会了权限模型（角色+资源可见性+审计）和安全细节（密码哈希、密钥不泄露、路径注入防护）。

5.2 改进建议

1. 内存限制跨平台：当前 Windows 下 `RLIMIT_AS` 失效，可改用 Docker 沙箱或 `cgroups` 统一评测环境，兼顾安全与一致。
2. 持久化升级：JSON 文件在小规模可用，但并发写入有覆盖风险，可迁移到 SQLite。
3. 评测隔离：用户代码在服务器本地进程运行，存在一定安全风险，生产环境应引入容器/沙箱。
4. 前端体验：Streamlit 的轮询式交互在长评测下体验一般，可考虑 WebSocket 推送。

5.3 时间投入

累计投入约 20-30 小时，其中需求分析与接口对齐约 4 小时、后端编码约 8-10 小时、前端约 4-6 小时、自测与修 bug 约 4-6 小时、文档约 2 小时。